

Generated GraphQL API Guide

- [Generated GraphQL API Guide](#)
 - [1. One endpoint per Platform](#)
 - [Available Platform implementations](#)
 - [2. Where the GraphQL surface comes from](#)
 - [3. Naming conventions](#)
 - [4. Scalar mapping](#)
 - [5. Mutations](#)
 - [5.1 From an Aggregate/](#)
 - [5.2 From a StateChangeSlice/ \(DCB\)](#)
 - [5.3 From an InboundTranslationSlice/](#)
 - [5.4 Suppressing mutation fields](#)
 - [6. Queries](#)
 - [6.1 Single-entity query and type](#)
 - [6.2 Connection query \(Relay\)](#)
 - [6.3 Items query \(range over a sub-id\)](#)
 - [6.4 Index queries](#)
 - [7. Filters and sorting \(annotation-driven\)](#)
 - [8. Cross-table references and labels](#)
 - [Cross-table references — @resolves and @resolvesMany](#)
 - [Composing display labels — @displayName](#)
 - [9. Platform-wide types](#)
 - [10. Subscriptions](#)
 - [10.1 Source A — Event-log feed — on<EventLog>EventLog_eventAppended](#)
 - [10.2 Source B — Read-model feed — on<Type> stateChanged\(id: ID\)](#)
 - [10.3 Source C — Mutation-accepted feed — on<MutationField>\(id: ID\)](#)
 - [11. Worked example: onLine-shop-hybrid](#)
 - [Platform scaffolding](#)
 - [Entity types](#)
 - [Connection plumbing \(shown for Catalog_Category; same pattern for every entity\)](#)
 - [Subscription supporting types](#)
 - [Mutations](#)
 - [Queries](#)
 - [Subscriptions](#)

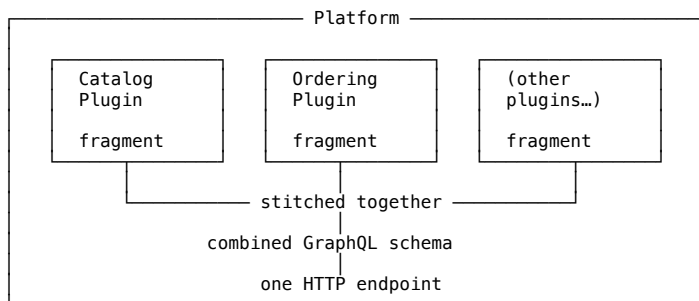
Generated GraphQL API Guide

This guide describes the GraphQL API that Reventless derives from your plugin specs. It is for application developers who want to know exactly what types, queries, mutations, and subscriptions appear in the schema, and which PPX annotations control them. The `onLine-shop-hybrid` example (Catalog + Ordering plugins) is the running illustration.

1. One endpoint per Platform

Every Reventless Platform exposes a **single GraphQL endpoint**. There is no per-plugin endpoint and no per-aggregate endpoint — clients always talk to the platform.

Each Plugin in the platform contributes a **GraphQL fragment**: a self-contained piece of SDL that declares the plugin's own types, queries, mutations, and subscriptions. The platform then **stitches all plugin fragments together** into one combined schema and serves it from the single endpoint:



Stitching means:

- the union of all type definitions across fragments is kept (collisions on the same type name are detected and reported as warnings — pick distinct names);
- the Query, Mutation, and Subscription root types are merged so that every plugin's fields coexist as siblings;
- the platform-wide scaffolding (Node, PageInfo, SortOrder, node(id), scalar AWSJSON, directive @aws_subscribe) is added once, regardless of how many plugins are installed.

Scope of this guide: the Domain API only. The Domain API is what your application clients consume — the types, queries, mutations, and subscriptions derived from your plugins' specs. The platform also exposes a separate Admin API for framework-level introspection and management (heartbeats, schema metadata, replay control); Admin-side fields are out of scope here.

Available Platform implementations

Reventless currently ships two Platform implementations. The same plugin code, the same @schema declarations, and the same generated SDL run on both — the only thing that changes is where the schema is served and how the underlying infrastructure is provisioned.

Implementation	Package	Endpoint	Backing infrastructure	Typical use
In-memory Platform	reventless-in-memory	http://localhost:4000/graphql (graphql-yoga) with GraphQL enabled	All components run in one Node process; event log, query db, and bus are JS data structures	Local development, integration tests, demos
AWS Platform	reventless-aws	AppSync HTTPS endpoint provisioned by Pulumi	DynamoDB (event log + query db), Lambda (command handlers, projections), SQS / SNS (event/command bus), S3 (task buckets), AppSync (GraphQL gateway)	Production deployments

Both platforms expose the same Domain API shape described in this guide. The differences clients may notice in practice:

- **Subscriptions.** The framework generates three subscription feeds for every platform (covered in §10):
 - **Source A** — event-log feed (on<EventLog>EventLog_eventAppended): one field per event log, fires on each appended event.
 - **Source B** — read-model feed (on<Type>_stateChanged): one field per queryable type, fires after the projection has written the new state.
 - **Source C** — mutation-accepted feed (on<MutationField>): one field per mutation, fires when the mutation is accepted.

The in-memory platform delivers all three feeds over its in-process bus through graphql-yoga's subscription transport. The AWS platform delivers Source C via AppSync's native @aws_subscribe directive and Sources A and B via AppSync Events.

- **Authorization.** The AWS platform honours per-table Cognito authorization rules attached at the read-model level; the in-memory platform ignores them (every operation is permitted).
- **Latency and durability.** Operations on the in-memory platform are synchronous and lose state when the process exits; the AWS platform is eventually consistent (event → projection → query db) and durable.

Use the in-memory platform while you build and test your plugins, and switch to the AWS platform for deployment — without changing any plugin code.

2. Where the GraphQL surface comes from

Every type and field in the schema is derived from one of:

- a @schema declaration in a component spec (Aggregate/, ReadModel/, StateChangeSlice/, StateViewSlice/, StateViewSliceStream/, InboundTranslationSlice/);
- a field-level annotation on a state schema that shapes filtering and sorting (@id, @compositeId, @subId, @compositeSubId, @index, @indexSubId, @scan, @scanSort, @displayName);
- a tag annotation on a command/event/state field that promotes a String scalar to an ID (@s.matches(Reventless.DcbTag, string)) — auto-applied inside *Slice/ folders).

Components whose role is purely internal contribute **no GraphQL surface**:

- AutomationSlice/
- OutboundTranslationSlice/
- ExtensionPoint/ (the spec and mapping files)
- Extension/
- Task/

If you need any of these to be reachable from a client, expose the entry point through a StateChangeSlice (mutation) or StateViewSlice/ReadModel (query) in the same plugin.

3. Naming conventions

Mutation and query field names, and the GraphQL type names they return, are always prefixed with the plugin name and an underscore. Pluralization is rule-based (–y → –ies, sibilant endings → –es, otherwise –s).

Component	Mutation field	Query field (single)	Query field (list)	GraphQL type name
Aggregate/Category.res	Catalog_Category_<Variant> —	—	—	—
StateChangeSlice/AddProduct.res	Catalog_AddProduct	—	—	—
ReadModel/Categories.res	—	Catalog_Category	Catalog_Categories	Catalog_Category
StateViewSliceStream/Products.res	—	Catalog_Product	Catalog_Products	Catalog_Product
StateViewSliceStream/ProductDemand.res	—	Catalog_ProductDemand	Catalog_ProductDemands	Catalog_ProductDemand
ReadModel/CatalogActivity.res	—	Catalog_CatalogActivity	Catalog_CatalogActivities	Catalog_CatalogActivity

Connection plumbing (<Type>Edge, <Type>Connection, <Type>Filter, <Type>OrderField, <Type>OrderBy) attaches to the GraphQL type name — e.g. Catalog_CategoryEdge, Catalog_CategoryConnection, Catalog_CategoryFilter, Catalog_CategoryOrderField, Catalog_CategoryOrderBy.

4. Scalar mapping

surly type	GraphQL type	Notes
string	String!	Plain text.
string + @s.matches(Reventless.DcbTag, string)	ID!	Entity-id tag → ID scalar.
float	Float!	
int	Int!	

surly type	GraphQL type	Notes
bool	Boolean!	
bigint	String!	Stringified for JSON safety.
array<T>	[T]!	
option<T> / Null(T)	T	The ! is dropped — field is nullable.
record { ... } declared with @schema	type X { ... }	Each record produces one named GraphQL type.
closed variant of records	sibling object types	One nested type per variant arm.
payload-less variant (X	Y Z)	

! (non-null) is applied to every field unless the source schema is wrapped in option(...) / Null(...). This matches the expectations of Relay clients and AppSync.

5. Mutations

The return type of every mutation is String! — the resolver returns a JSON-encoded outcome (command id on success, structured error otherwise). Domain errors are **not** modelled as a typed GraphQL union.

5.1 From an Aggregate/

A closed union command produces one mutation field per variant. The field name is <Plugin>_<Entity>_<Variant>, and an id: ID! argument is **prepended** to the variant's native args (because aggregates are keyed by the aggregate id).

```
// catalog/src/Category/Aggregate/Category.res
@@reventless.spec
```

```
@schema
type command =
  | Add({name: string})
  | Rename({name: string})
  | Archive
```

```
extend type Mutation {
  Catalog_Category_Add(id: ID!, name: String!): String!
  Catalog_Category_Rename(id: ID!, name: String!): String!
  Catalog_Category_Archive(id: ID!): String!
}
```

5.2 From a StateChangeSlice/ (DCB)

A single-object command produces **one** mutation field. The slice file name becomes the field name. The command's record fields become the mutation arguments — **no id is prepended**, because the command already carries every key it needs (its DCB tags).

```
// catalog/src/Product/StateChangeSlice/AddProduct.res
@@reventless.spec
```

```
@schema
type command =
  | AddProduct({
    productId: string,
    name: string,
    description: string,
    price: float,
  })
```

```
extend type Mutation {
  Catalog_AddProduct(productId: ID!, name: String!, description: String!, price: Float!): String!
}
```

(productId becomes ID! because, inside *Slice/ folders, the PPX automatically applies the DCB tag matcher to *Id fields.)

Multi-key commands work the same way. Ordering_PlaceOrder carries an order id, a customer id, and a list of product ids:

```
@schema
type command =
  PlaceOrder({@partitionTag orderId: string, customerId: string, productIds: array<string>})
```

```
Ordering_PlaceOrder(orderId: ID!, customerId: ID!, productIds: [ID!]!): String!
```

A StateChangeSlice may also use a union command, in which case it emits one field per variant (still without a prepended id):

```
// ordering/src/CatalogProduct/StateChangeSlice/SyncCatalogProduct.res
@schema
type command =
  | SyncNewProduct({productId: string, name: string, price: float})
  | ChangeSyncedPrice({productId: string, price: float})
```

```
Ordering_SyncCatalogProduct_SyncNewProduct(productId: ID!, name: String!, price: Float!): String!
Ordering_SyncCatalogProduct_ChangeSyncedPrice(productId: ID!, price: Float!): String!
```

5.3 From an InboundTranslationSlice/

An inbound translation slice exposes a mutation whose argument list comes from @schema type externalInput — **not** the internal command. The slice's translation function maps the external shape into the internal command before publishing it.

```
// catalog/src/Product/InboundTranslationSlice/ImportProduct.res
@@reventless.spec
```

```
@schema
type externalInput = {
  sku: string,
  title: string,
  desc: string,
  unitPrice: int,
  currency: string,
}
```

```
Catalog_ImportProduct(sku: String!, title: String!, desc: String!, unitPrice: Int!, currency: String!): String!
```

Use this when an external system needs to call you with their vocabulary and you translate to your domain command.

5.4 Suppressing mutation fields

Annotation

Effect

`@noApi` on the command type The whole mutation field (or all variants) is omitted.
`@noApiVariants(["Variant1", ...])` Only the listed union variants are omitted; others remain.

6. Queries

Every `ReadModel`, `StateViewSlice`, and `StateViewSliceStream` produces a queryable type, and the type is treated uniformly for SDL purposes:

- it implements `Node`;
- it has an `id`: `ID!` field injected as the first field;
- the single-entity query accepts (`id`: `ID!`) (plus a sort-key argument if the state has a `@subId`);
- a Relay-style connection query is generated alongside;
- an additional `<Single>Items` connection is generated when there is a `@subId`;
- an additional `<Single>By<IndexName>` connection is generated for each `@index`.

The state schema's own fields appear **alongside** the injected `id` — for example, the `Catalog_Product` type below has both `id`: `ID!` (synthetic) and `productId`: `ID!` (from the state schema). Clients can fetch by either, but only `id` participates in `Node`.

6.1 Single-entity query and type

```
// catalog/src/Category/ReadModel/Categories.res
@@reventless.spec
```

```
@schema
type state = {
  name: string,
  archived: bool,
}
```

```
type Catalog_Category implements Node {
  id: ID!
  name: String!
  archived: Boolean!
}
```

```
extend type Query {
  Catalog_Category(id: ID!): Catalog_Category
}
```

A `StateViewSliceStream` follows the same shape — the only practical difference is that the state schema typically carries its own entity `id` field:

```
// catalog/src/Product/StateViewSliceStream/Products.res
@schema
type state = {productId: string, name: string, description: string, price: float}
```

```
type Catalog_Product implements Node {
  id: ID!
  productId: ID!
  name: String!
  description: String!
  price: Float!
}
```

```
extend type Query {
  Catalog_Product(id: ID!): Catalog_Product
}
```

6.2 Connection query (Relay)

For every queryable type the framework also emits a Relay-style connection. Using `Catalog_Category` as a worked example:

```
type Catalog_CategoryEdge {
  node: Catalog_Category!
  cursor: String!
}
```

```

type Catalog_CategoryConnection {
  edges: [Catalog_CategoryEdge!]!
  pageInfo: PageInfo!
}

input Catalog_CategoryFilter {
  search: String
  searchPrefix: String
  ids: [ID!]
  # plus per-field filters derived from @id / @subId / @index / @scan (see §7)
}

enum Catalog_CategoryOrderField {
  # one value per @id / @subId / @index / @scanSort field on the state
}

input Catalog_CategoryOrderBy {
  field: Catalog_CategoryOrderField!
  direction: SortOrder!
}

extend type Query {
  Catalog_Categories(
    filter: Catalog_CategoryFilter,
    orderBy: Catalog_CategoryOrderBy,
    first: Int, after: String,
    last: Int, before: String
  ): Catalog_CategoryConnection!
}

```

When the state has no sortable fields, the OrderField enum is empty (or omitted) and the orderBy argument disappears with it.

PageInfo, Node, and SortOrder are platform-wide types (see §9).

6.3 Items query (range over a sub-id)

When the state has a @subId field, you get an additional <Single>Items connection that ranges *within* one parent id, and the single-entity query gains the sort-key argument:

```

@schema
type state = {
  @id productId: string,
  @subId createdAt: string,
  name: string,
}

extend type Query {
  Catalog_Product(id: ID!, createdAt: String!): Catalog_Product
  Catalog_ProductItems(
    id: ID!,
    filter: Catalog_ProductItemsFilter,
    first: Int, after: String, last: Int, before: String
  ): Catalog_ProductConnection!
}

```

Catalog_ProductItemsFilter is a sibling of Catalog_ProductFilter that restricts the range filters to the sub-id field.

6.4 Index queries

Each @index field produces a dedicated connection query named <Single>By<IndexName>. The framework strips a leading lowercase by from the index name before prepending its own By:

@index argument Resulting query field name

```

~name="byOwner" <Single>ByOwner
~name="owner"    <Single>ByOwner
~name="0wner"   <Single>ByOwner
~name="ByOwner" <Single>ByByOwner (avoid)

```

Pick byOwner (lowercase by prefix). PascalCase ByOwner is *not* stripped and produces the doubled form.

```

@schema
type state = {
  productId: string,
  @index(~name="byOwner") ownerId: string,
  name: string,
}

extend type Query {
  Catalog_Product(id: ID!): Catalog_Product
  Catalog_ProductByOwner(
    ownerId: ID,
    first: Int, after: String, last: Int, before: String
  ): Catalog_ProductConnection!
}

```

7. Filters and sorting (annotation-driven)

The `<Type>Filter` input and `<Type>OrderField` enum on every connection are built from the state's field annotations.

State annotation	Filter inputs added	Sortable	Notes
@id	<field>Eq	yes	Primary key.
@compositeId (≥ 2 fields)	<field>Eq for each	no	Compound primary key.
@subId	<field>Eq, <field>From, <field>To	yes	Sort key — enables range filters and items query.
@compositeSubId (≥ 2 fields)	<field>Eq, <field>From, <field>To	yes	Compound sort key.
@index / @index(~name="...")	<field>Eq	yes	Emits <Single>By<Name> connection query.
@indexSubId("<Name>")	range filters scoped to the named index	yes	Sort key for the named @index.
@scan	<field>Eq	no	Opt-in equality without a backing index.
@scanSort	—	yes	Opt-in sortable without a backing sort key.
(no annotation)	not filterable	no	

The base filter always includes:

```
search: String
searchPrefix: String
ids: [ID!]
```

@scan and @scanSort are deliberately verbose — they enable client filtering and sorting on fields that have no backing index. On AWS DynamoDB they cost O(n); on the in-memory platform they are free. Use them where ergonomics outweigh cost.

8. Cross-table references and labels

Annotation	Effect on SDL
@resolves({table, field, via?, plugin?, sourceSubId?, subIdArg??})	On a string field. Adds a virtual field of the target entity type alongside the original id field.
@resolvesMany({table, field, plugin??})	On an array<string> field. Adds a virtual list field of the target entity type alongside the original id list.
@displayName / @displayName("sep") (≥ 1 field)	Composes a synthetic displayName: String field on the generated type, derived by joining the annotated fields.

Cross-table references — @resolves and @resolvesMany

The two annotations let your query expand an id (or list of ids) into the full target entity, without requiring the client to make a second round trip.

- `table` — the target table name (matches the target plugin's spec name, e.g. the singular form "Customers" for the Customers ReadModel).
- `field` — the name of the **virtual** field that will appear on the source type. Choose any name that fits your domain ("customer", "products", "placedBy").
- `via` (*optional, single only*) — the name of a target-side @index to resolve through, instead of the primary key.
- `plugin` (*optional*) — the target plugin's name, when it lives in a different plugin. Omit for same-plugin references.

Example: order references its customer and the products it contains

```
// ordering/src/Order/StateViewSlice/Orders.res
@@reventless.spec

@schema
type status = Placed | Shipped | Cancelled

@schema
type state = {
  orderId: string,
  @resolves({table: "Customers", field: "customer"})
  customerId: string,
  @resolvesMany({table: "AvailableProducts", field: "products"})
  productIds: array<string>,
  status: status,
}
```

The generated GraphQL type keeps the original id fields **and** adds the resolved entities:

```
type Ordering_Order implements Node {
  id: ID!
  orderId: ID!

  customerId: ID!           # original id stays
  customer: Ordering_Customer # virtual – resolved on read

  productIds: [ID!]         # original id list stays
  products: [Ordering_AvailableProduct] # virtual – resolved on read

  status: Ordering_OrderStatus!
}
```

Clients can now fetch:

```

query OrderDetails($id: ID!) {
  Ordering_Order(id: $id) {
    orderId
    status
    customer {
      email
      displayName
    }
    products {
      name
      price
    }
  }
}

```

Notes on usage:

- The original customerId/productIds fields are **not** removed — clients that only need ids stay efficient.
- The resolved entity type must be queryable in its own right (i.e. the target plugin exposes it as a ReadModel or StateViewSlice).
- When the target lives in another plugin, add plugin: "<TargetPlugin>":

```

@resolves({plugin: "Catalog", table: "Products", field: "product"})
productId: string,

```

- When the target table has a composite key and you want to resolve through a secondary index, set via: "<indexName>":

```

@resolves({table: "Customers", field: "owner", via: "byOwnerId"})
ownerId: string,

```

Composing display labels — @displayName

@displayName composes a synthetic displayName: String! field on the generated type by joining one or more annotated state fields. With a single annotated field the synthetic field mirrors it; with multiple, the values are joined by " " (or the separator you pass).

```

@schema
type state = {
  @displayName email: string,
  address: string,
  deactivated: bool,
}

```

```

type Ordering_Customer implements Node {
  id: ID!
  email: String!
  address: String!
  deactivated: Boolean!
  displayName: String!
}

```

The displayName field is automatically populated whenever a projection writes the row, so clients never need to assemble the label themselves.

9. Platform-wide types

The schema always begins with these scaffolding declarations, regardless of which plugins you ship:

```

scalar AWSJSON

directive @aws_subscribe(mutations: [String!]) on FIELD_DEFINITION

interface Node {
  id: ID!
}

type PageInfo {
  hasNextPage: Boolean!
  hasPreviousPage: Boolean!
  startCursor: String
  endCursor: String
}

enum SortOrder { ASC DESC }

type Query {
  node(id: ID!): Node
  # ... extended by each plugin
}

```

If no plugin contributes any mutations, the schema declares type Mutation { _noop: String } so it still parses. The same fallback applies to subscriptions.

@aws_subscribe is an AppSync directive used by the mutation-accepted subscription feed (§10.3). On the AWS deployment it routes subscription delivery through AppSync; on the in-memory platform it is a no-op.

10. Subscriptions

There are three subscription feeds. Each is generated automatically from the same specs that drive mutations and queries.

10.1 Source A — Event-log feed — on<EventLog>EventLog_eventAppended

One field per aggregate EventLog and per plugin DcbEventLog. Fires when any event is appended to that log. Carries the raw event as AWSJSON. Primarily for observability and admin tooling.

```
type CatalogEventLogEvent {
  position: String!
  eventType: String!
  payload: AWSJSON!
  originatorSlice: String
}

extend type Subscription {
  onCatalogEventLog_eventAppended: CatalogEventLogEvent
  onOrderingEventLog_eventAppended: OrderingEventLogEvent
  onCategoryEventLog_eventAppended: CategoryEventLogEvent
  onCustomerEventLog_eventAppended: CustomerEventLogEvent
}
```

originatorSlice is set when the event was appended through a StateChangeSlice (so you can attribute change in mixed-source projections).

10.2 Source B — Read-model feed — on<Type>_stateChanged(id: ID)

One field per queryable type. Fires *after* the projection has fully written the new state to the QueryDb. The optional id argument is a server-side filter — clients receive only updates for the entity they care about.

```
extend type Subscription {
  onCatalog_Category_stateChanged(id: ID): Catalog_Category
  onCatalog_Product_stateChanged(id: ID): Catalog_Product
  onOrdering_Order_stateChanged(id: ID): Ordering_Order
  onOrdering_Customer_stateChanged(id: ID): Ordering_Customer
}
```

Use this to drive live UI updates.

10.3 Source C — Mutation-accepted feed — on<MutationField>(id: ID)

One field per mutation. Fires when the mutation is accepted (before its side effects complete). Useful for optimistic UI progress indicators.

```
extend type Subscription {
  onCatalog_AddProduct(id: ID): String!
    @aws_subscribe(mutations: ["Catalog_AddProduct"])
  onCatalog_Category_Add(id: ID): String!
    @aws_subscribe(mutations: ["Catalog_Category_Add"])
  onOrdering_PlaceOrder(id: ID): String!
    @aws_subscribe(mutations: ["Ordering_PlaceOrder"])
}
```

AppSync delivers these subscriptions natively via the @aws_subscribe directive — no additional infrastructure is needed.

11. Worked example: online-shop-hybrid

This section shows the complete GraphQL surface a client sees against the hybrid platform (Catalog + Ordering). To keep the SDL readable, the per-type connection plumbing (Edge, Connection, Filter, OrderField, OrderBy) is shown in full only for Catalog_Category; every other entity follows the same pattern with the appropriate type-name prefix.

Platform scaffolding

```
scalar AWSJSON
directive @aws_subscribe(mutations: [String!]) on FIELD_DEFINITION

interface Node { id: ID! }

type PageInfo {
  hasNextPage: Boolean!
  hasPreviousPage: Boolean!
  startCursor: String
  endCursor: String
}

enum SortOrder { ASC DESC }

type Query { node(id: ID!): Node }
type Mutation { _noop: String }
type Subscription { _noop: String }
```

Query, Mutation, and Subscription are then extended by each plugin.

Entity types

```
# Catalog
type Catalog_Category implements Node {
  id: ID!
```



```

    name: String!
    archived: Boolean!
}

type Catalog_Product implements Node {
  id: ID!
  productId: ID!
  name: String!
  description: String!
  price: Float!
}

type Catalog_ProductDemand implements Node {
  id: ID!
  productId: ID!
  name: String!
  orderCount: Int!
}

enum Catalog_CatalogActivityKind { Category Product }
enum Catalog_CatalogActivityChange { Added Renamed Archived }

type Catalog_CatalogActivity implements Node {
  id: ID!
  name: String!
  kind: Catalog_CatalogActivityKind!
  lastChange: Catalog_CatalogActivityChange!
}

# Ordering
type Ordering_Customer implements Node {
  id: ID!
  email: String!
  address: String!
  deactivated: Boolean!
  displayName: String!
}

enum Ordering_OrderStatus { Placed Shipped Cancelled }

type Ordering_Order implements Node {
  id: ID!
  orderId: ID!
  customerId: ID!
  productIds: [ID!]
  status: Ordering_OrderStatus!
}

type Ordering_AvailableProduct implements Node {
  id: ID!
  productId: ID!
  name: String!
  price: Float!
}

```

Connection plumbing (shown for Catalog_Category; same pattern for every entity)

```

type Catalog_CategoryEdge { node: Catalog_Category! cursor: String! }
type Catalog_CategoryConnection { edges: [Catalog_CategoryEdge!]! pageInfo: PageInfo! }
input Catalog_CategoryFilter { search: String searchPrefix: String ids: [ID!] }
enum Catalog_CategoryOrderField { name }
input Catalog_CategoryOrderBy { field: Catalog_CategoryOrderField! direction: SortOrder! }

```

Repeat for every entity type with the appropriate prefix (Catalog_Product*, Catalog_ProductDemand*, Catalog_CatalogActivity*, Ordering_Customer*, Ordering_Order*, Ordering_AvailableProduct*). The OrderField enum values come from state fields tagged @id/@subId/@index/@scanSort; absent any such tags, the enum is empty and the orderBy argument is dropped.

Subscription supporting types

```

type CatalogEventLogEvent { position: String! eventType: String! payload: AWSJSON! originatorSlice: String }
type OrderingEventLogEvent { position: String! eventType: String! payload: AWSJSON! originatorSlice: String }
type CategoryEventLogEvent { position: String! eventType: String! payload: AWSJSON! originatorSlice: String }
type CustomerEventLogEvent { position: String! eventType: String! payload: AWSJSON! originatorSlice: String }

```

Mutations

```

extend type Mutation {
  # Catalog - Category aggregate
  Catalog_Category_Add(id: ID!, name: String!): String!
  Catalog_Category_Rename(id: ID!, name: String!): String!
  Catalog_Category_Archive(id: ID!): String!

  # Catalog - Product DCB slices
  Catalog_AddProduct(productId: ID!, name: String!, description: String!, price: Float!): String!
  Catalog_ChangeProductName(productId: ID!, newName: String!): String!
  Catalog_ChangeProductDescription(productId: ID!, description: String!): String!
}

```

```

Catalog_ChangeProductPrice(productId: ID!, price: Float!): String!
Catalog_RecordProductDemand(productId: ID!): String!

# Catalog – Inbound translation (external schema as args)
Catalog_ImportProduct(sku: String!, title: String!, desc: String!, unitPrice: Int!, currency: String!): String!

# Ordering – Customer aggregate
Ordering_Customer_Register(id: ID!, email: String!, address: String!): String!
Ordering_Customer_UpdateEmail(id: ID!, email: String!): String!
Ordering_Customer_UpdateAddress(id: ID!, address: String!): String!
Ordering_Customer_Deactivate(id: ID!): String!

# Ordering – Order DCB slices
Ordering_PlaceOrder(orderId: ID!, customerId: ID!, productIds: [ID!]!): String!
Ordering_ShipOrder(orderId: ID!): String!
Ordering_CancelOrder(orderId: ID!): String!
Ordering_RefundOrder(orderId: ID!): String!

# Ordering – CatalogProduct sync (driven by the Extension; also reachable directly)
Ordering_SyncCatalogProduct_SyncNewProduct(productId: ID!, name: String!, price: Float!): String!
Ordering_SyncCatalogProduct_ChangeSyncedPrice(productId: ID!, price: Float!): String!
}

```

Queries

```

extend type Query {
  # Catalog
  Catalog_Category(id: ID!): Catalog_Category
  Catalog_Categories(filter: Catalog_CategoryFilter, orderBy: Catalog_CategoryOrderBy, first: Int, after: String, last: Int, before: String): Catalog_CategoryConnection!

  Catalog_Product(id: ID!): Catalog_Product
  Catalog_Products(filter: Catalog_ProductFilter, orderBy: Catalog_ProductOrderBy, first: Int, after: String, last: Int, before: String): Catalog_ProductConnection!

  Catalog_ProductDemand(id: ID!): Catalog_ProductDemand
  Catalog_ProductDemands(filter: Catalog_ProductDemandFilter, orderBy: Catalog_ProductDemandOrderBy, first: Int, after: String, last: Int, before: String): Catalog_ProductDemandConnection!

  Catalog_CatalogActivity(id: ID!): Catalog_CatalogActivity
  Catalog_CatalogActivities(filter: Catalog_CatalogActivityFilter, orderBy: Catalog_CatalogActivityOrderBy, first: Int, after: String, last: Int, before: String): Catalog_CatalogActivityConnection!

  # Ordering
  Ordering_Customer(id: ID!): Ordering_Customer
  Ordering_Customers(filter: Ordering_CustomerFilter, orderBy: Ordering_CustomerOrderBy, first: Int, after: String, last: Int, before: String): Ordering_CustomerConnection!

  Ordering_Order(id: ID!): Ordering_Order
  Ordering_Orders(filter: Ordering_OrderFilter, orderBy: Ordering_OrderOrderBy, first: Int, after: String, last: Int, before: String): Ordering_OrderConnection!

  Ordering_AvailableProduct(id: ID!): Ordering_AvailableProduct
  Ordering_AvailableProducts(filter: Ordering_AvailableProductFilter, orderBy: Ordering_AvailableProductOrderBy, first: Int, after: String, last: Int, before: String): Ordering_AvailableProductConnection!
}

```

Subscriptions

```

extend type Subscription {
  # Mutation-accepted feed (one per mutation field)
  onCatalog_Category_Add(id: ID): String! @aws_subscribe(mutations: ["Catalog_Category_Add"])
  onCatalog_AddProduct(id: ID): String! @aws_subscribe(mutations: ["Catalog_AddProduct"])
  onOrdering_PlaceOrder(id: ID): String! @aws_subscribe(mutations: ["Ordering_PlaceOrder"])
  # ... one entry per mutation above

  # Read-model feed (one per queryable type)
  onCatalog_Category_stateChanged(id: ID): Catalog_Category
  onCatalog_Product_stateChanged(id: ID): Catalog_Product
  onCatalog_ProductDemand_stateChanged(id: ID): Catalog_ProductDemand
  onCatalog_CatalogActivity_stateChanged(id: ID): Catalog_CatalogActivity
  onOrdering_Customer_stateChanged(id: ID): Ordering_Customer
  onOrdering_Order_stateChanged(id: ID): Ordering_Order
  onOrdering_AvailableProduct_stateChanged(id: ID): Ordering_AvailableProduct

  # Event-log feed (one per event log)
  onCatalogEventLog_eventAppended: CatalogEventLogEvent
  onOrderingEventLog_eventAppended: OrderingEventLogEvent
  onCategoryEventLog_eventAppended: CategoryEventLogEvent
  onCustomerEventLog_eventAppended: CustomerEventLogEvent
}

```

Catalog publishes one DcbEventLog (covering all Product / ProductDemand slices) plus the Category aggregate's own log. Ordering publishes one DcbEventLog (covering all Order / CatalogProduct slices) plus the Customer aggregate's own log.